

Solo-Git Phase 2: Safe Refactor Plan

Generated: November 3, 2025

Branch: phase-2-audit-refactor

Principle: **NO BEHAVIOR CHANGES** - Only structural improvements

Table of Contents

1. [Refactoring Principles](#)
 2. [Safe Refactoring Checklist](#)
 3. [Phase 2A: Test Infrastructure](#)
 4. [Phase 2B: Code Consolidation](#)
 5. [Phase 2C: Function Extraction](#)
 6. [Phase 2D: Naming Standardization](#)
 7. [Phase 2E: Documentation](#)
 8. [Deferred Changes](#)
 9. [Commit Strategy](#)
 10. [Validation Strategy](#)
-

Refactoring Principles

What We CAN Do (Safe Refactoring)

✓ **Structural Changes:**

- Extract functions for better readability
- Consolidate duplicate code
- Rename internal functions (not CLI commands)
- Add type hints
- Add docstrings
- Reorganize imports
- Create base classes/utilities

✓ **Test Infrastructure:**

- Add new tests
- Improve test coverage
- Add test utilities
- Create test fixtures

✓ **Documentation:**

- Add missing docstrings
- Update architecture docs
- Create developer guides

What We CANNOT Do (Behavior Changes)

✗ External API Changes:

- Change CLI command names
- Change CLI flags or arguments
- Change JSON output formats
- Change configuration file format
- Change state file format

✗ Functional Changes:

- Modify business logic
- Change error messages (user-facing)
- Change default behaviors
- Add new features

✗ Data Changes:

- Modify state schema
 - Change database structure
 - Alter file formats
-

Safe Refactoring Checklist

Before any refactoring:

- ☐ All existing tests pass
- ☐ No public API changes
- ☐ No CLI command changes
- ☐ No user-facing behavior changes
- ☐ Changes are backwards compatible

After refactoring:

- ☐ All existing tests still pass
 - ☐ New tests added for refactored code
 - ☐ Documentation updated
 - ☐ Type hints added where possible
 - ☐ Code review completed
-

Phase 2A: Test Infrastructure

Goal: Achieve 90%+ coverage on critical modules before refactoring them

Duration: 3 days (~22 hours)

Risk: Low - Only adding tests, no code changes

A1: Configuration Tests (~4 hours)

Target: `sologit/config/manager.py`

Current Coverage: ~85% (estimated)

Target Coverage: 95%

Test Plan

File: tests/test_config_manager_comprehensive.py

```

"""Comprehensive tests for ConfigManager."""
import pytest
import tempfile
import os
from pathlib import Path
from sologit.config.manager import ConfigManager, SoloGitConfig

class TestConfigManager:
    """Test ConfigManager functionality."""

    @pytest.fixture
    def temp_config_dir(self):
        """Create temporary config directory."""
        with tempfile.TemporaryDirectory() as tmpdir:
            yield Path(tmpdir)

    def test_load_default_config(self, temp_config_dir):
        """Test loading default configuration."""
        manager = ConfigManager(config_dir=temp_config_dir)
        config = manager.load_config()

        assert config is not None
        assert config.models is not None
        assert config.budget is not None

    def test_load_from_yaml(self, temp_config_dir):
        """Test loading configuration from YAML file."""
        config_file = temp_config_dir / "config.yaml"
        config_file.write_text("""
abacus:
  api_key: test_key
  endpoint: https://api.example.com
  """

        manager = ConfigManager(config_file=config_file)
        config = manager.load_config()

        assert config.abacus.api_key == "test_key"

    def test_env_var_override(self, temp_config_dir, monkeypatch):
        """Test environment variable overrides."""
        monkeypatch.setenv("ABACUS_API_KEY", "env_key")

        manager = ConfigManager(config_dir=temp_config_dir)
        config = manager.load_config()

        assert config.abacus.api_key == "env_key"

    def test_profile_switching(self, temp_config_dir):
        """Test loading different profiles."""
        # Create dev profile
        dev_config = temp_config_dir / "config.dev.yaml"
        dev_config.write_text("""
budget:
  daily_usd_cap: 5.0
  """

        # Create prod profile
        prod_config = temp_config_dir / "config.prod.yaml"
        prod_config.write_text("""
budget:
  daily_usd_cap: 50.0

```

```

    """

    manager = ConfigManager(config_dir=temp_config_dir)

    dev = manager.load_profile("dev")
    assert dev.budget.daily_usd_cap == 5.0

    prod = manager.load_profile("prod")
    assert prod.budget.daily_usd_cap == 50.0

    def test_config_validation(self, temp_config_dir):
        """Test configuration validation."""
        config_file = temp_config_dir / "config.yaml"
        config_file.write_text("""
budget:
  daily_usd_cap: -10 # Invalid: negative
""")

        manager = ConfigManager(config_file=config_file)

        with pytest.raises(ValueError, match="daily_usd_cap must be positive"):
            manager.load_config()

    def test_config_defaults(self, temp_config_dir):
        """Test default values are applied."""
        manager = ConfigManager(config_dir=temp_config_dir)
        config = manager.load_config()

        # Should have default values
        assert config.budget.daily_usd_cap == 10.0
        assert config.models.planning_model == "gpt-4o"

    def test_save_config(self, temp_config_dir):
        """Test saving configuration."""
        config_file = temp_config_dir / "config.yaml"

        manager = ConfigManager(config_file=config_file)
        config = manager.load_config()
        config.budget.daily_usd_cap = 25.0

        manager.save_config(config)

        # Reload and verify
        new_manager = ConfigManager(config_file=config_file)
        new_config = new_manager.load_config()
        assert new_config.budget.daily_usd_cap == 25.0

    def test_config_encryption(self, temp_config_dir):
        """Test credential encryption (future feature)."""
        pytest.skip("Encryption not yet implemented")

```

Commit: test: Add comprehensive ConfigManager tests

A2: State Management Tests (~6 hours)

Target: `sologit/state/manager.py`, `sologit/state/git_sync.py`

Current Coverage: ~77%

Target Coverage: 90%

Test Plan

File: tests/test_state_manager_comprehensive.py

```

"""Comprehensive tests for StateManager."""
import pytest
from sologit.state.manager import StateManager, JSONStateBackend
from sologit.state.schema import WorkpadStatus

class TestStateManager:
    """Test StateManager functionality."""

    @pytest.fixture
    def state_manager(self, tmp_path):
        """Create StateManager with temporary directory."""
        return StateManager(state_dir=tmp_path)

    def test_create_repository(self, state_manager):
        """Test repository creation in state."""
        repo_id = state_manager.create_repository(
            repo_id="test-repo",
            path="/path/to/repo",
            trunk="main"
        )

        assert repo_id == "test-repo"
        repo = state_manager.get_repository("test-repo")
        assert repo is not None
        assert repo.trunk == "main"

    def test_create_workpad(self, state_manager):
        """Test workpad creation."""
        # Create repository first
        state_manager.create_repository("repo1", "/path", "main")

        # Create workpad
        workpad_id = state_manager.create_workpad(
            repo_id="repo1",
            workpad_id="pad1",
            title="test feature",
            base_commit="abc123"
        )

        assert workpad_id == "pad1"
        workpad = state_manager.get_workpad("repo1", "pad1")
        assert workpad.title == "test feature"
        assert workpad.status == WorkpadStatus.ACTIVE

    def test_state_persistence(self, state_manager):
        """Test state is persisted to disk."""
        state_manager.create_repository("repo1", "/path", "main")

        # Create new manager pointing to same directory
        new_manager = StateManager(state_dir=state_manager.state_dir)

        # Should load existing state
        repo = new_manager.get_repository("repo1")
        assert repo is not None

    def test_state_corruption_recovery(self, state_manager, tmp_path):
        """Test recovery from corrupted state file."""
        # Create valid state
        state_manager.create_repository("repo1", "/path", "main")

        # Corrupt the state file
        state_file = tmp_path / "state.json"

```

```

state_file.write_text("CORRUPTED{{{")

# Should recover gracefully
new_manager = StateManager(state_dir=tmp_path)
repos = new_manager.list_repositories()

# Should start with empty state (backup created)
assert len(repos) == 0

def test_concurrent_state_access(self, state_manager):
    """Test concurrent state access."""
    import threading

    errors = []

    def create_workpad(i):
        try:
            state_manager.create_repository(f"repo{i}", f"/path{i}", "main")
        except Exception as e:
            errors.append(e)

    threads = [threading.Thread(target=create_workpad, args=(i,)) for i in range(10)]

    for t in threads:
        t.start()
    for t in threads:
        t.join()

    assert len(errors) == 0
    assert len(state_manager.list_repositories()) == 10

def test_state_backup_on_save(self, state_manager, tmp_path):
    """Test backup is created on state save."""
    state_manager.create_repository("repo1", "/path", "main")

    backup_dir = tmp_path / "backups"
    assert backup_dir.exists()
    assert len(list(backup_dir.glob("*.json"))) > 0

```

File: tests/test_git_sync_comprehensive.py


```

"""Comprehensive tests for GitStateSync."""
import pytest
from sologit.state.git_sync import GitStateSync

class TestGitStateSync:
    """Test Git-State synchronization."""

    @pytest.fixture
    def git_repo(self, tmp_path):
        """Create test Git repository."""
        import subprocess
        repo_path = tmp_path / "test-repo"
        repo_path.mkdir()

        subprocess.run(["git", "init"], cwd=repo_path, check=True)
        subprocess.run(["git", "config", "user.email", "test@test.com"], cwd=repo_path)

        subprocess.run(["git", "config", "user.name", "Test User"], cwd=repo_path)

        # Create initial commit
        (repo_path / "README.md").write_text("# Test")
        subprocess.run(["git", "add", "."], cwd=repo_path, check=True)
        subprocess.run(["git", "commit", "-m", "Initial commit"], cwd=repo_path,
check=True)

        return repo_path

    def test_detect_new_commits(self, git_repo):
        """Test detection of new commits."""
        sync = GitStateSync(git_repo)

        # Initial sync
        sync.sync_from_git()
        initial_commits = sync.get_commits()

        # Create new commit
        (git_repo / "file.txt").write_text("test")
        import subprocess
        subprocess.run(["git", "add", "."], cwd=git_repo, check=True)
        subprocess.run(["git", "commit", "-m", "Add file"], cwd=git_repo, check=True)

        # Sync again
        sync.sync_from_git()
        new_commits = sync.get_commits()

        assert len(new_commits) == len(initial_commits) + 1

    def test_bidirectional_sync(self, git_repo):
        """Test Git ↔ State synchronization."""
        sync = GitStateSync(git_repo)

        # Sync from Git to State
        sync.sync_from_git()

        # Modify state
        sync.add_workpad("pad1", "feature", "abc123")

        # Sync back to Git
        sync.sync_to_git()

        # Verify Git has workpad branch
        import subprocess

```

```
result = subprocess.run(
    ["git", "branch", "--list", "pads/pad1"],
    cwd=git_repo,
    capture_output=True,
    text=True
)
assert "pads/pad1" in result.stdout
```

Commits:

- test: Add comprehensive StateManager tests
 - test: Add GitStateSync bidirectional sync tests
-

A3: CLI Command Tests (~8 hours)

Target: `sologit/cli/commands.py`, `sologit/cli/ci_commands.py`

Current Coverage: ~35%

Target Coverage: 75%

Test Plan

File: `tests/test_cli_commands_integration.py`

```

"""Integration tests for CLI commands."""
import pytest
from click.testing import CliRunner
from sologit.cli.commands import (
    pad_create, pad_list, pad_info, pad_promote, pad_delete,
    repo_init_zip, repo_list, test_run
)

class TestCLICommands:
    """Test CLI command execution."""

    @pytest.fixture
    def runner(self):
        """Create Click test runner."""
        return CliRunner()

    @pytest.fixture
    def test_repo(self, runner, tmp_path):
        """Create test repository."""
        # Setup test repo
        pass

    def test_pad_create_success(self, runner, test_repo):
        """Test successful workpad creation."""
        result = runner.invoke(pad_create, ["my-feature"])

        assert result.exit_code == 0
        assert "Created workpad" in result.output
        assert "my-feature" in result.output

    def test_pad_create_invalid_name(self, runner, test_repo):
        """Test workpad creation with invalid name."""
        result = runner.invoke(pad_create, ["invalid/name/with/slashes"])

        assert result.exit_code == 1
        assert "Error" in result.output

    def test_pad_list_empty(self, runner, test_repo):
        """Test listing workpads when none exist."""
        result = runner.invoke(pad_list)

        assert result.exit_code == 0
        assert "No workpads" in result.output

    def test_pad_list_with_workpads(self, runner, test_repo):
        """Test listing multiple workpads."""
        # Create workpads
        runner.invoke(pad_create, ["feature-1"])
        runner.invoke(pad_create, ["feature-2"])

        # List workpads
        result = runner.invoke(pad_list)

        assert result.exit_code == 0
        assert "feature-1" in result.output
        assert "feature-2" in result.output

    def test_pad_info(self, runner, test_repo):
        """Test workpad info display."""
        runner.invoke(pad_create, ["my-feature"])

        result = runner.invoke(pad_info, ["my-feature"])

```

```

    assert result.exit_code == 0
    assert "my-feature" in result.output
    assert "Status:" in result.output

def test_pad_promote_happy_path(self, runner, test_repo):
    """Test successful workpad promotion."""
    runner.invoke(pad_create, ["my-feature"])

    # Make some changes and commit
    # ... (test setup)

    result = runner.invoke(pad_promote, ["my-feature"])

    assert result.exit_code == 0
    assert "Promoted" in result.output

def test_pad_delete(self, runner, test_repo):
    """Test workpad deletion."""
    runner.invoke(pad_create, ["my-feature"])

    result = runner.invoke(pad_delete, ["my-feature"])

    assert result.exit_code == 0
    assert "Deleted" in result.output

    # Verify it's gone
    list_result = runner.invoke(pad_list)
    assert "my-feature" not in list_result.output

def test_json_output_format(self, runner, test_repo):
    """Test JSON output format."""
    runner.invoke(pad_create, ["my-feature"])

    result = runner.invoke(pad_list, ["--json"])

    assert result.exit_code == 0

    import json
    data = json.loads(result.output)
    assert isinstance(data, list)
    assert len(data) == 1
    assert data[0]["id"] == "my-feature"

def test_verbose_flag(self, runner, test_repo):
    """Test verbose output."""
    result = runner.invoke(pad_list, ["-v"])

    assert result.exit_code == 0
    # Verbose output should include more details

def test_error_handling(self, runner, test_repo):
    """Test error message formatting."""
    result = runner.invoke(pad_info, ["nonexistent"])

    assert result.exit_code == 1
    assert "Error" in result.output
    assert "not found" in result.output.lower()

```

Commits:

- test: Add CLI integration tests for workpad commands

- test: Add CLI integration tests for repository commands
 - test: Add CLI output format tests (JSON, CSV)
-

A4: AI Provider Adapter Tests (~4 hours)

Target: `sologit/orchestration/providers/*_adapter.py`

Current Coverage: ~85%

Target Coverage: 95%

Test Plan

File: `tests/test_providers_comprehensive.py`

```

"""Comprehensive tests for AI provider adapters."""
import pytest
from unittest.mock import Mock, patch, AsyncMock
from sologit.orchestration.providers.abacus_adapter import AbacusAdapter
from sologit.orchestration.providers.openai_adapter import OpenAIAdapter
from sologit.orchestration.providers.anthropic_adapter import AnthropicAdapter
from sologit.orchestration.providers import ProviderConfig, ProviderType

class TestAbacusAdapter:
    """Test Abacus.AI adapter."""

    @pytest.fixture
    def adapter(self):
        """Create Abacus adapter."""
        config = ProviderConfig(
            provider_type=ProviderType.ABACUS,
            api_key="test_key"
        )
        return AbacusAdapter(config)

    @pytest.mark.asyncio
    async def test_generate_success(self, adapter):
        """Test successful generation."""
        with patch.object(adapter.client, 'chat_completion') as mock_chat:
            mock_chat.return_value = {
                "content": "Generated text",
                "model": "gpt-4o",
                "usage": {"total_tokens": 50},
                "cost_usd": 0.001
            }

            response = await adapter.generate(
                prompt="Test prompt",
                system_prompt="You are a helpful assistant"
            )

            assert response.content == "Generated text"
            assert response.provider == ProviderType.ABACUS
            assert response.tokens_used == 50

    @pytest.mark.asyncio
    async def test_generate_network_error(self, adapter):
        """Test handling of network errors."""
        with patch.object(adapter.client, 'chat_completion') as mock_chat:
            mock_chat.side_effect = ConnectionError("Network error")

            with pytest.raises(ConnectionError):
                await adapter.generate(prompt="Test")

    @pytest.mark.asyncio
    async def test_generate_rate_limit(self, adapter):
        """Test handling of rate limits."""
        with patch.object(adapter.client, 'chat_completion') as mock_chat:
            mock_chat.side_effect = Exception("Rate limit exceeded")

            with pytest.raises(Exception, match="Rate limit"):
                await adapter.generate(prompt="Test")

    def test_is_available_success(self, adapter):
        """Test availability check when API is reachable."""
        with patch.object(adapter.client, 'ping') as mock_ping:
            mock_ping.return_value = True

```

```

        assert adapter.is_available() is True

    def test_is_available_failure(self, adapter):
        """Test availability check when API is unreachable."""
        with patch.object(adapter.client, 'ping') as mock_ping:
            mock_ping.side_effect = Exception("Connection failed")

            assert adapter.is_available() is False

    def test_default_model(self, adapter):
        """Test default model selection."""
        assert adapter.get_default_model() == "routellm-auto"

# Similar tests for OpenAI and Anthropic adapters...

```

Commits:

- test: Add comprehensive Abacus adapter tests
- test: Add comprehensive OpenAI adapter tests
- test: Add comprehensive Anthropic adapter tests
- test: Add provider failover integration tests

Phase 2A Summary

Duration: 3 days (~22 hours)**Deliverables:**

- [] 40+ new tests across critical modules
- [] Coverage increased from 76% to ~85%
- [] All critical paths tested
- [] Test utilities and fixtures created

Validation:

```

# Run new tests
pytest tests/test_config_manager_comprehensive.py -v
pytest tests/test_state_manager_comprehensive.py -v
pytest tests/test_cli_commands_integration.py -v
pytest tests/test_providers_comprehensive.py -v

# Check coverage
pytest --cov=sologit --cov-report=html tests/

```

Phase 2B: Code Consolidation

Goal: Eliminate duplicate code without changing behavior**Duration:** 1 day (~6 hours)**Risk:** Low - Pure code movement

B1: Consolidate CLI Utilities (~2 hours)

Current Duplication

Function `abort_with_error` appears in 4 files:

- `cli/main.py`
- `cli/config_commands.py`
- `cli/commands.py`
- `cli/integrated_commands.py`

Refactoring

Create: `sologit/cli/utils.py`

```

"""CLI utility functions."""
import sys
from typing import Optional, NoReturn
from rich.console import Console

console = Console()

def abort_with_error(message: str, code: int = 1) -> NoReturn:
    """Print error message and exit.

    Args:
        message: Error message to display
        code: Exit code (default: 1)

    Raises:
        SystemExit: Always exits with specified code
    """
    console.print(f"[red]Error:[/red] {message}")
    sys.exit(code)

def success_message(message: str) -> None:
    """Print success message.

    Args:
        message: Success message to display
    """
    console.print(f"[green]✓[/green] {message}")

def warning_message(message: str) -> None:
    """Print warning message.

    Args:
        message: Warning message to display
    """
    console.print(f"[yellow]⚠[/yellow] {message}")

def info_message(message: str) -> None:
    """Print info message.

    Args:
        message: Info message to display
    """
    console.print(f"[cyan]i[/cyan] {message}")

```

Update: All CLI files to use `from sologit.cli.utils import abort_with_error`

Test: `tests/test_cli_utils.py`


```

"""Tests for CLI utilities."""
import pytest
from sologit.cli.utils import abort_with_error, success_message

def test_abort_with_error():
    """Test abort_with_error exits with correct code."""
    with pytest.raises(SystemExit) as exc_info:
        abort_with_error("Test error", code=2)

    assert exc_info.value.code == 2

def test_success_message(capsys):
    """Test success message output."""
    success_message("Test success")
    captured = capsys.readouterr()
    assert "Test success" in captured.out

```

Commits:

- refactor: Create CLI utils module
- refactor: Consolidate abort_with_error into cli/utils
- test: Add CLI utils tests

B2: Consolidate TUI Actions (~2 hours)

Current Duplication

TUI actions (`action_help` , `action_refresh`) appear in 3 files:

- `ui/enhanced_tui.py`
- `ui/tui_app.py`
- `ui/heaven_tui.py`

Refactoring

Create: `sologit/ui/base_tui.py`

```

"""Base TUI actions and utilities."""
from textual.app import App
from textual.widgets import Static

class BaseTUIActions:
    """Base class for common TUI actions."""

    def action_help(self) -> None:
        """Show help overlay.

        Displays keyboard shortcuts and available commands.
        """
        self.push_screen("help")

    def action_refresh(self) -> None:
        """Refresh current view.

        Reloads data and updates all widgets.
        """
        self.refresh()

    def action_quit(self) -> None:
        """Quit the application.

        Saves state and exits gracefully.
        """
        self.exit()

class BaseTUI(App, BaseTUIActions):
    """Base TUI application with common functionality."""

    BINDINGS = [
        ("q", "quit", "Quit"),
        ("?", "help", "Help"),
        ("r", "refresh", "Refresh"),
    ]

```

Update: All TUI classes to inherit from `BaseTUI`

Commits:

- refactor: Create base TUI actions class
- refactor: Migrate TUI classes to use BaseTUI
- test: Add BaseTUI action tests

B3: Create Test Utilities (~2 hours)

Create Shared Test Fixtures

Create: `tests/conftest.py`

```

"""Shared test fixtures and utilities."""
import pytest
import tempfile
from pathlib import Path
from click.testing import CliRunner
from sologit.engines.git_engine import GitEngine
from sologit.state.manager import StateManager

@pytest.fixture
def temp_dir():
    """Create temporary directory."""
    with tempfile.TemporaryDirectory() as tmpdir:
        yield Path(tmpdir)

@pytest.fixture
def git_repo(temp_dir):
    """Create test Git repository."""
    import subprocess
    repo_path = temp_dir / "test-repo"
    repo_path.mkdir()

    subprocess.run(["git", "init"], cwd=repo_path, check=True)
    subprocess.run(["git", "config", "user.email", "test@test.com"], cwd=repo_path)
    subprocess.run(["git", "config", "user.name", "Test User"], cwd=repo_path)

    # Create initial commit
    (repo_path / "README.md").write_text("# Test")
    subprocess.run(["git", "add", "."], cwd=repo_path, check=True)
    subprocess.run(["git", "commit", "-m", "Initial commit"], cwd=repo_path, check=True)

    return repo_path

@pytest.fixture
def git_engine(git_repo):
    """Create GitEngine with test repository."""
    return GitEngine(repo_path=git_repo)

@pytest.fixture
def state_manager(temp_dir):
    """Create StateManager with temporary directory."""
    return StateManager(state_dir=temp_dir)

@pytest.fixture
def cli_runner():
    """Create Click test runner."""
    return CliRunner()

@pytest.fixture
def mock_api_client():
    """Create mock Abacus API client."""
    from unittest.mock import Mock
    client = Mock()
    client.chat_completion.return_value = {
        "content": "Generated text",
        "model": "gpt-4o",
        "usage": {"total_tokens": 50},
        "cost_usd": 0.001
    }
    return client

```

Commit: test: Add shared test fixtures and utilities

Phase 2B Summary

Duration: 1 day (~6 hours)

Deliverables:

- [] cli/utils.py - CLI utility functions
- [] ui/base_tui.py - Base TUI actions
- [] tests/conftest.py - Shared test fixtures
- [] Eliminated ~200 lines of duplicate code

Validation:

```
# Run all tests to ensure no breakage
pytest tests/ -v

# Check for remaining duplication
grep -r "def abort_with_error" sologit/
# Should only find it in cli/utils.py
```

Phase 2C: Function Extraction

Goal: Break large functions into smaller, testable units

Duration: 4 days (~30 hours)

Risk: Medium - Requires careful testing

C1: Extract execute_pair_loop (~4 hours)

Target: sologit/cli/commands.py::execute_pair_loop (283 lines)

Current Structure

One massive function handling entire AI pairing workflow.

Target Structure

```

# cli/commands.py

def execute_pair_loop(
    prompt: str,
    context: Dict,
    config: SoloGitConfig,
    workpad_id: Optional[str] = None
) -> PairResult:
    """Execute AI pair programming loop.

    Args:
        prompt: User's request
        context: Repository context
        config: Configuration
        workpad_id: Optional workpad to use

    Returns:
        PairResult with success/failure and details
    """
    # Step 1: Parse prompt
    task = _parse_pair_prompt(prompt, context)

    # Step 2: Select or create workpad
    workpad = _get_or_create_workpad(workpad_id, task, config)

    # Step 3: Plan changes with AI
    plan = _plan_changes_with_ai(task, context, config)

    # Step 4: Generate patches
    patches = _generate_patches_from_plan(plan, config)

    # Step 5: Apply patches to workpad
    apply_result = _apply_patches_to_workpad(workpad, patches)

    # Step 6: Run tests
    test_result = _run_workpad_tests(workpad, config)

    # Step 7: Handle test results
    if test_result.passed:
        return _handle_success(workpad, test_result)
    else:
        return _handle_failure(workpad, test_result, config)

def _parse_pair_prompt(prompt: str, context: Dict) -> Task:
    """Parse user prompt into structured task.

    Args:
        prompt: Raw user input
        context: Repository context

    Returns:
        Structured Task object
    """
    # Extract task type, files, requirements
    return Task(
        type=_infer_task_type(prompt),
        description=prompt,
        files=_extract_mentioned_files(prompt, context),
        requirements=_extract_requirements(prompt)
    )

def _get_or_create_workpad(

```

```

    workpad_id: Optional[str],
    task: Task,
    config: SoloGitConfig
) -> Workpad:
    """Get existing workpad or create new one.

    Args:
        workpad_id: Optional existing workpad ID
        task: Task to execute
        config: Configuration

    Returns:
        Workpad object
    """
    if workpad_id:
        return _get_existing_workpad(workpad_id)
    else:
        return _create_workpad_for_task(task, config)

def _plan_changes_with_ai(
    task: Task,
    context: Dict,
    config: SoloGitConfig
) -> CodePlan:
    """Use AI to plan code changes.

    Args:
        task: Task to plan
        context: Repository context
        config: Configuration

    Returns:
        CodePlan with proposed changes
    """
    orchestrator = AIOrchestrator(config)
    return orchestrator.plan(task.description, context)

# ... more extracted functions

```

Benefits of Extraction

- Each function is independently testable
- Clear inputs and outputs
- Single responsibility
- Easy to mock for testing
- Reusable components

Testing Strategy

```
# tests/test_pair_loop_components.py

def test_parse_pair_prompt():
    """Test prompt parsing."""
    prompt = "Add user authentication to auth.py"
    context = {"files": ["auth.py", "models.py"]}

    task = _parse_pair_prompt(prompt, context)

    assert task.type == TaskType.FEATURE
    assert "auth.py" in task.files
    assert "authentication" in task.description.lower()

def test_get_or_create_workpad_existing(mock_git_engine):
    """Test getting existing workpad."""
    workpad = _get_or_create_workpad(
        workpad_id="existing-pad",
        task=None,
        config=None
    )

    assert workpad.id == "existing-pad"

def test_get_or_create_workpad_new(mock_git_engine):
    """Test creating new workpad."""
    task = Task(type=TaskType.FEATURE, description="Add auth")

    workpad = _get_or_create_workpad(
        workpad_id=None,
        task=task,
        config=Config()
    )

    assert workpad is not None
    assert "auth" in workpad.title.lower()

# ... more component tests
```

Commits (small, incremental):

1. refactor: Extract `_parse_pair_prompt` from `execute_pair_loop`
2. test: Add tests for `_parse_pair_prompt`
3. refactor: Extract `_get_or_create_workpad`
4. test: Add tests for `_get_or_create_workpad`
5. refactor: Extract `_plan_changes_with_ai`
6. test: Add tests for `_plan_changes_with_ai`
7. refactor: Complete `execute_pair_loop` extraction
8. test: Add integration test for `execute_pair_loop`

C2-C5: Additional Large Function Extractions

Following the same pattern, extract:

- **C2:** `cli/commands.py::test_run` (252 lines) → ~3 hours
- **C3:** `cli/commands.py::generate_commit_message` (201 lines) → ~2 hours

- **C4:** workflows/auto_merge.py::execute (183 lines) → ~4 hours
- **C5:** orchestration/code_generator.py::generate_patch (141 lines) → ~2 hours

Each extraction follows:

1. Identify logical sections
2. Extract into private functions
3. Add type hints
4. Add docstrings
5. Write unit tests for each extracted function
6. Write integration test for main function

Phase 2C Summary

Duration: 4 days (~30 hours)

Deliverables:

- [] 5 large functions refactored into ~30 smaller functions
- [] ~60 new unit tests for extracted components
- [] ~100% test coverage on refactored functions
- [] Improved maintainability and testability

Validation:

```
# Run all tests
pytest tests/ -v --cov=sologit

# Verify no large functions remain
python3 << 'EOF'
import ast
from pathlib import Path

for py_file in Path("sologit").rglob("*.py"):
    if "__pycache__" in str(py_file):
        continue
    with open(py_file) as f:
        tree = ast.parse(f.read())
    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            if hasattr(node, 'end_lineno'):
                size = node.end_lineno - node.lineno
                if size > 100:
                    print(f"{py_file}:{node.lineno} {node.name}: {size} lines")

EOF
```

Phase 2D: Naming Standardization

Goal: Consistent naming across codebase

Duration: 0.5 day (~4 hours)

Risk: Low - Pure renames

D1: Standardize Data Retrieval Functions (~2 hours)

Naming Convention

Operation	Verb	Usage
Get in-memory object	get_*	get_workpad(id)
Fetch from external API	fetch_*	fetch_ai_response(prompt)
Read from file/disk	read_*	read_config_file(path)
Load from database	load_*	load_user_data(id)

Renames Required

Category 1: Retrieval → Get

- retrieve_workpad() → get_workpad()
- obtain_config() → get_config()

Category 2: Consolidate Read/Load

- load_config() → Keep (file I/O)
- read_state() → Keep (file I/O)
- get_config() → Keep (in-memory)

Implementation

```
# Use automated refactoring tools
# Example with Python rope library:

python3 << 'EOF'
from rope.base.project import Project
from rope.refactor.rename import Rename

project = Project("sologit")

# Rename function
resource = project.root.get_file("sologit/some_file.py")
offset = # calculate offset to function name
rename = Rename(project, resource, offset)
changes = rename.get_changes("new_name")
project.do(changes)
EOF
```

Testing Strategy:

```
# After each rename:
1. Run all tests: pytest tests/ -v
2. Verify no broken imports: python -m sologit
3. Run type checker: mypy sologit/
```

Commits:

- refactor: Standardize get_* naming convention
- refactor: Standardize fetch_* naming convention

- refactor: Standardize read_* naming convention
- docs: Update naming conventions guide

D2: Standardize Error Handling (~2 hours)

Current State

Mixed error handling patterns:

- Some functions return `None`
- Some raise generic `RuntimeError`
- Some use custom exceptions

Target State

Consistent exception hierarchy:

```
# sologit/core/exceptions.py

class SoloGitError(Exception):
    """Base exception for Solo-Git."""
    pass

class WorkpadError(SoloGitError):
    """Workpad-related errors."""
    pass

class WorkpadNotFoundError(WorkpadError):
    """Workpad not found."""
    pass

class WorkpadAlreadyExistsError(WorkpadError):
    """Workpad already exists."""
    pass

class RepositoryError(SoloGitError):
    """Repository-related errors."""
    pass

class StateError(SoloGitError):
    """State management errors."""
    pass

class AIError(SoloGitError):
    """AI orchestration errors."""
    pass
```

Refactoring

Replace:

```
# Before
def get_workpad(workpad_id: str):
    if not exists(workpad_id):
        return None # ❌ Silent failure
    return load_workpad(workpad_id)
```

With:

```
# After
def get_workpad(workpad_id: str) -> Workpad:
    """Get workpad by ID.

    Args:
        workpad_id: Workpad identifier

    Returns:
        Workpad object

    Raises:
        WorkpadNotFoundError: If workpad doesn't exist
    """
    if not exists(workpad_id):
        raise WorkpadNotFoundError(f"Workpad '{workpad_id}' not found")
    return load_workpad(workpad_id)
```

Commits:

- refactor: Create exception hierarchy
- refactor: Replace None returns with exceptions
- refactor: Replace generic exceptions with specific ones
- test: Add exception handling tests

Phase 2D Summary

Duration: 0.5 day (~4 hours)**Deliverables:**

- [] Consistent naming conventions applied
- [] Exception hierarchy implemented
- [] All `return None` replaced with exceptions
- [] Naming convention guide created

Phase 2E: Documentation

Goal: Document all refactored code**Duration:** 1 day (~8 hours)**Risk:** Low - Pure documentation

E1: Add Missing Docstrings (~4 hours)

Target: 86 undocumented functions**Template:**

```
def function_name(param1: Type1, param2: Type2) -> ReturnType:
    """Brief one-line description.

    Longer description explaining what the function does,
    when to use it, and any important caveats.

    Args:
        param1: Description of param1
        param2: Description of param2

    Returns:
        Description of return value

    Raises:
        ExceptionType: When this exception is raised

    Examples:
        >>> function_name("value1", 123)
        ReturnType(...)
    """
    pass
```

Priority Order:

1. Public API functions (high impact)
2. Internal utilities (medium impact)
3. Private helpers (low impact)

Commits:

- docs: Add docstrings to API client methods
- docs: Add docstrings to CLI commands
- docs: Add docstrings to state management
- docs: Add docstrings to orchestration modules

E2: Update Architecture Documentation (~2 hours)**Files to Update:**

- ARCHITECTURE.md - Reflect new structure
- README.md - Update component descriptions
- docs/wiki/architecture/ - Update diagrams

Changes:

- Document new utility modules
- Update function extraction diagrams
- Clarify component boundaries

Commit: docs: Update architecture documentation

E3: Create Developer Guides (~2 hours)

Create: docs/CONTRIBUTING.md

Contributing to Solo-Git

Code Standards

Naming Conventions

- Use ``get_*` for in-memory retrieval
- Use ``fetch_*` for external API calls
- Use ``read_*` for file I/O
- Use ``create_*` for object creation
- Use ``build_*` for constructed objects

Error Handling

- Always use specific exceptions (never generic ``Exception``)
- Document raised exceptions in docstrings
- Never return ``None`` to indicate errors

Function Size

- Keep functions under 100 lines
- Extract logical sections into private functions
- Single responsibility per function

Testing

- Write tests before refactoring
- Aim for 90%+ coverage on new code
- Use fixtures from ``tests/conftest.py``

Commits

- Small, focused commits
- Follow conventional commit format
- Reference issue numbers

Create: docs/DEVELOPMENT.md

Development Guide

Setup Development Environment

```
```bash
```

#### # Clone repository

```
git clone https://github.com/yourusername/solo-git
cd solo-git
```

#### # Create virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

#### # Install in development mode

```
pip install -e .[dev,test]
```

#### # Run tests

```
pytest tests/ -v
```

## Running Tests

---

```
All tests
pytest tests/

With coverage
pytest --cov=sologit tests/

Specific module
pytest tests/test_git_engine.py

With verbose output
pytest tests/ -v -s
```

## Code Quality

---

```
Format code
black solokit/ tests/

Sort imports
isort solokit/ tests/

Type checking
mypy solokit/

Linting
ruff check solokit/ tests/
```

## Making Changes

---

1. Create feature branch: `git checkout -b feature/my-feature`
2. Make changes
3. Add tests
4. Run test suite
5. Commit with conventional commits
6. Create pull request

```

Commits:
- ☐ docs: Create contributing guide
- ☐ docs: Create development guide

Phase 2E Summary

Duration: 1 day (~8 hours)
Deliverables:
- [] 86 functions documented
- [] Architecture documentation updated
- [] Contributing guide created
- [] Development guide created

Deferred Changes

These changes would alter external behavior and are DEFERRED to future phases:

Deferred to Phase 3 (Feature Development)
- ☒ GitHub Actions integration
- ☒ Security scanning hooks
- ☒ Deployment simulation
- ☒ Notification system
- ☒ Credential encryption
- ☒ Command aliases
- ☒ GUI write operations completion

Deferred to Phase 4 (Advanced Features)
- ☒ Advanced caching
- ☒ Performance optimizations
- ☒ Plugin system
- ☒ Multi-repository support

Commit Strategy

Commit Message Format

Follow [Conventional Commits](https://www.conventionalcommits.org/):

```

():

```

Types:
- `feat:` New feature
- `fix:` Bug fix
- `refactor:` Code refactoring (no behavior change)
- `test:` Adding tests
- `docs:` Documentation changes
- `style:` Code style changes (formatting, etc.)
- `chore:` Maintenance tasks

Examples:

```



refactor(cli): Extract large functions into smaller units - Extract `_parse_pair_prompt` from `execute_pair_loop` - Extract `_get_or_create_workpad` - Extract `_plan_changes_with_ai` - Improves testability and maintainability Refs: #123

test(config): Add comprehensive ConfigManager tests - Test YAML loading - Test environment variable overrides - Test profile switching - Test validation - Increases coverage from 85% to 95%

```
Commit Frequency

- **Small, frequent commits** (every 30-60 minutes)
- **One logical change per commit**
- **Tests in same commit** as refactored code
- **Documentation in same commit** as code changes
```

#### ### Commit Review

```
Before committing:
- [] All tests pass
- [] Code formatted (black, isort)
- [] No type errors (mypy)
- [] No lint errors (ruff)
- [] Documentation updated
- [] Commit message follows convention
```

```

```

#### ### Validation Strategy

##### ### After Each Refactoring Step

```
```bash
# 1. Run all tests
pytest tests/ -v

# 2. Check coverage
pytest --cov=sologit --cov-report=html tests/

# 3. Type checking
mypy solokit/

# 4. Linting
ruff check solokit/ tests/

# 5. Format check
black --check solokit/ tests/
isort --check solokit/ tests/

# 6. Run smoke tests
python -m solokit.cli.main hello
python -m solokit.cli.main version
```

After Phase Completion

```
# 1. Full test suite
pytest tests/ -v --cov=sologit --cov-report=html

# 2. Integration tests
pytest tests/ -v -m integration

# 3. Performance tests (if any)
pytest tests/ -v -m performance

# 4. Manual smoke testing
# - Create workpad
# - Run tests
# - Promote workpad
# - Check state
```

Before PR Creation

```
# 1. Rebase on main
git fetch origin
git rebase origin/main

# 2. Run full validation
make preflight # (to be created)

# 3. Generate reports
make audit # (to be created)

# 4. Review changes
git log --oneline origin/main..HEAD
git diff origin/main
```

--- ## Risk Mitigation ### Backup Strategy Before starting refactoring:

```
# Create backup branch
git checkout -b phase-2-audit-refactor-backup
git checkout phase-2-audit-refactor
```

Rollback Plan If refactoring introduces issues:

```
# Identify problematic commit
git log --oneline

# Revert specific commit
git revert <commit-hash>

# Or reset to previous state
git reset --hard <good-commit>

# Or restore from backup
git checkout phase-2-audit-refactor-backup
git checkout -b phase-2-audit-refactor-v2
```

Testing Safeguards - ****Never skip tests**** - Always run full suite - ****Test in isolation**** - Use fresh virtual environment - ****Test on clean state**** - Clear caches and temp files - ****Test with real data**** - Use actual repositories for smoke tests --- ## Progress Tracking ### Daily Checklist ****Morning****: - [] Pull latest changes - [] Review plan for the day - [] Run baseline tests ****During Work****: - [] Commit

every 30-60 minutes - [] Run tests after each commit - [] Update documentation as you go ****Evening****: - [] Review commits - [] Run full test suite - [] Update progress in audit report - [] Push to remote **### Weekly Review** - [] Coverage improvements - [] Refactoring completed - [] Documentation updated - [] Technical debt reduced --- **## Success Criteria ### Phase 2 is Complete When**: - [] ****Test Coverage****: 85%+ overall, 95%+ on critical modules - [] ****Code Quality****: No functions >100 lines - [] ****Consistency****: All naming conventions applied - [] ****Documentation****: All public functions documented - [] ****Validation****: All tests passing - [] ****No Regressions****: Contract tests prove no behavior changes --- **## Timeline Summary**

Phase	Duration	Focus
2A: Test Infrastructure	3 days	Add tests to critical modules
2B: Code Consolidation	1 day	Eliminate duplicates
2C: Function Extraction	4 days	Break large functions
2D: Naming Standardization	0.5 day	Consistent naming
2E: Documentation	1 day	Add docstrings and guides
Total	**9.5 days**	**~75 hours**

****With buffer for testing and validation****: ****10-12 days total**** --- **## Next Steps** 1. ****Review this plan**** with team/stakeholders 2. ****Create GitHub issues**** for each phase 3. ****Set up project board**** for tracking 4. ****Begin Phase 2A**** (Test Infrastructure) 5. ****Daily standups**** to track progress 6. ****Weekly demos**** of improvements --- ***This refactor plan is a living document and will be updated as work progresses.***